

Das Halteproblem und andere unentscheidbare Probleme

The Halting Problem and other undecidable Problems

Andreas Schäfer

Theoretische Informatik

TH Lübeck

- (1) Jeder hat das Recht, seine Meinung in Wort, Schrift und Bild frei zu äußern und zu verbreiten und sich aus allgemein zugänglichen Quellen ungehindert zu unterrichten. Die Pressefreiheit und die Freiheit der Berichterstattung durch Rundfunk und Film werden gewährleistet. Eine Zensur findet nicht statt.*
- (2) Diese Rechte finden ihre Schranken in den Vorschriften der allgemeinen Gesetze, den gesetzlichen Bestimmungen zum Schutze der Jugend und in dem Recht der persönlichen Ehre.*
- (3) Kunst und Wissenschaft, Forschung und Lehre sind frei. Die Freiheit der Lehre entbindet nicht von der Treue zur Verfassung.*

Artikel 5 Grundgesetz

Bitte evaluieren Sie das Modul!



```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    puts("Hello World\n");
    return EXIT_SUCCESS;
}
```

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int total = 3;
    int x, y, z;
    while (1) {
        for (x = 1; x < total - 2; x++) {
            for (y = 1; y < total - x - 1; y++) {
                z = total - x - y;
                if (x * x * x + y * y * y == z * z * z) {
                    puts("Hello World\n");
                    return EXIT_SUCCESS;
                }
            }
        }
        total++;
    }
}
```

Ja	Nein (in Praxis doch)
☐	☒

Gibt das Programm „Hello World aus“?

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int total = 3;
    int x, y, z;
    while (1) {
        for (x = 1; x < total - 2; x++) {
            for (y = 1; y < total - x - 1; y++) {
                z = total - x - y;
                if (x * x * x + y * y * y == z * z * z) {
                    puts("Hello World\n");
                    return EXIT_SUCCESS;
                }
            }
        }
        total++;
    }
}
```

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    puts("Hello World\n");
    return EXIT_SUCCESS;
}
```

Ja Nein (in Praxis doch)



Kann das ein Programm prüfen?

- Ein Programm ist Eingabe in ein anderes Programm
- Anwendung:
 - IDEs prüfen, ob Programm Endlosschleife enthält
 - IDEs prüfen, ob eine bestimmte Anweisung erreichbar ist
 - IDEs prüfen, ob eine Variable einmal einen bestimmten Wert annimmt
Bsp. Pufferüberlauf, Bereichsprüfung

- Ein Programm ist Eingabe in ein anderes Programm
- Anwendung:
 - IDEs prüfen, ob Programm Endlosschleife enthält
 - IDEs prüfen, ob eine bestimmte Anweisung erreichbar ist
 - IDEs prüfen, ob eine Variable einmal einen bestimmten Wert annimmt
Bsp. Pufferüberlauf, Bereichsprüfung
- Frage heute: Wieso gibt es das noch nicht?

- Das Programm muss „kodiert“ sein
(Quellcode, Maschinencode, Bytecode)
- Unser Modell: Turing-Maschinen
- Wegen Churchscher These gelten die Resultate für alle Programme / Rechner.

- Eine TM ist ein Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$
- TM kann binär kodiert werden.
- Zu jeder natürlichen Zahl gibt es eine TM und jede TM ist durch mindestens eine Zahl kodiert
- Notation: M_w ist TM zu Kodierung $w \in \{0, 1\}^*$.

- Eine TM ist ein Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$
- TM kann binär kodiert werden.
- Zu jeder natürlichen Zahl gibt es eine TM und jede TM ist durch mindestens eine Zahl kodiert
- Notation: M_w ist TM zu Kodierung $w \in \{0, 1\}^*$.
- Eine Turing-Maschine kann eine andere Turing-Maschine simulieren,
- Turing-Maschine bekommt eine Codierung einer TM als Eingabe und simuliert diese TM.

Definition (Halten von TMs (engl. Halting))

Eine Turing-Maschine hält,

Definition (Halten von TMs (engl. Halting))

Eine Turing-Maschine hält, wenn Sie für aktuellen Zustand und aktuelles Bandzeichen keine Transition hat

Definition (Halten von TMs (engl. Halting))

Eine Turing-Maschine hält, wenn Sie für aktuellen Zustand und aktuelles Bandzeichen keine Transition hat

Definition (Akzeptanz durch Halten)

Eine Turing-Maschine M *akzeptiert* ein Wort w durch Halten, wenn die Turing-Maschine angesetzt auf das Wort irgendwann hält. $M \downarrow w$.

Definition (Akzeptanz durch Halten)

Eine Turing-Maschine M *akzeptiert* ein Wort w durch Halten, wenn die Turing-Maschine angesetzt auf das Wort irgendwann hält. $M \downarrow w$.

Definition (Akzeptanz durch Endzustand)

Eine Turing-Maschine *akzeptiert* ein Wort durch Endzustand, wenn die Turing-Maschine angesetzt auf das Wort in einen Endzustand kommen kann. Kurz: $(q_0 w) \rightarrow^* (\alpha q_f \beta)$, $q_f \in F$
(Konvention: In Endzustand hält die TM auf jeden Fall)

Bemerkung

Beide Akzeptanzbegriffe sind äquivalent

Von TM akzeptiert durch Halten zu TM akzeptiert durch Endzustand:

Alle undefinierten Übergänge werden in neuen Endzustand geleitet.

Von TM akzeptiert durch Endzustand zu TM akzeptiert durch Halten:

Alle undefinierten Übergänge außerhalb des Endzustands werden in neuen Nicht-Endzustand mit Endlosschleife geleitet.

Definition (Entscheidbarkeit)

Eine Sprache L (bzw. ein Problem) heißt *entscheidbar* (engl. decidable) genau dann wenn es eine Turing-Maschine M gibt, die angesetzt auf ein Wort w als Ergebnis 1 liefert wenn $w \in L$ gilt und sonst 0 liefert.

Definition (Spezielle Halteproblem)

Das spezielle Halteproblem ist das Problem ob eine Turing-Maschine angesetzt auf Ihre *eigene* Kodierung hält.

Formal: Das spezielle Halteproblem ist die Sprache

$$K = \{w \in \{0, 1\}^* \mid M_w \text{ angesetzt auf } w \text{ hält}\}$$

Definition (Spezielle Halteproblem)

Das spezielle Halteproblem ist das Problem ob eine Turing-Maschine angesetzt auf Ihre *eigene* Kodierung hält.

Formal: Das spezielle Halteproblem ist die Sprache

$$K = \{w \in \{0, 1\}^* \mid M_w \text{ angesetzt auf } w \text{ hält}\}$$

Wir betrachten jetzt Probleme mit Ja/Nein Antworten

Konvention: Probleme werden meist als Sprachen angegeben

Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.



Turing Maschine M entscheidet spezielles Halteproblem angesetzt auf Code w einer TM M_w

- liefert 0 falls M_w angesetzt auf eigenen Code w nicht hält
- liefert 1 falls M_w angesetzt auf eigenen Code w hält

Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.

Konstruiere M' aus M . Was passiert wenn man M' auf eigenen Code w' ansetzt? ☐

Turing Maschine M entscheidet spezielles Halteproblem angesetzt auf Code w einer TM M_w

- liefert 0 falls M_w angesetzt auf eigenen Code w nicht hält
- liefert 1 falls M_w angesetzt auf eigenen Code w hält

Turing Maschine M' aus M konstruiert

1. Simuliert M (Ergebnis steht auf Band)
2. Falls Band = 1 $\Rightarrow$ starte Endlosschleife, falls Band = 0 $\Rightarrow$ stop

Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.

Konstruiere M' aus M .

$$M' \text{ angesetzt auf } w' \text{ hält} \iff M \text{ angesetzt auf } w' \text{ liefert } 0$$



Turing Maschine M' aus M konstruiert

1. Simuliert M (Ergebnis steht auf Band)
2. Falls Band = 1 $\Rightarrow$ starte Endlosschleife, falls Band = 0 $\Rightarrow$ stop

Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.

Konstruiere M' aus M .

$$\begin{aligned} M' \text{ angesetzt auf } w' \text{ hält} &\iff M \text{ angesetzt auf } w' \text{ liefert } 0 \\ &\iff M_{w'} \text{ angesetzt auf } w' \text{ hält nicht} \end{aligned}$$



Turing Maschine M entscheidet spezielles Halteproblem angesetzt auf Code w einer TM M_w

- liefert 0 falls M_w angesetzt auf eigenen Code w nicht hält
- liefert 1 falls M_w angesetzt auf eigenen Code w hält

Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.

Konstruiere M' aus M . Was passiert wenn man M' auf eigenen Code w' ansetzt?

M' angesetzt auf w' hält $\iff M$ angesetzt auf w' liefert 0

$\iff M_{w'}$ angesetzt auf w' hält nicht

$\iff M'$ angesetzt auf w' hält nicht



Beweis.

Angenommen K ist entscheidbar. Dann gibt es eine TM M , die Problem K entscheidet.

Konstruiere M' aus M .

$$\begin{aligned} M' \text{ angesetzt auf } w' \text{ hält} &\iff M \text{ angesetzt auf } w' \text{ liefert 0} \\ &\iff M_{w'} \text{ angesetzt auf } w' \text{ hält nicht} \\ &\iff M' \text{ angesetzt auf } w' \text{ hält nicht} \end{aligned}$$

Also ist die Annahme falsch und das spezielle Halteproblem unentscheidbar. □

- Wir wollen weitere unentscheidbare Probleme finden
- Brauchen dazu einfache Methoden um Unentscheidbarkeit nachzuweisen
- Pizza - Problem

Wer kann Pizza backen?



Abbildung 1: Pizza (Quelle: Wikimedia Commons, CC-Lizenz: cyclonebill)

- Annahme: Sie haben einen Algorithmus für Thunfisch-Pizza
- Dann haben Sie auch einen Algorithmus für
 - Pizza Hawaii
 - Pizza Thunfisch-Diablo
 - Pizza Margherita

- Annahme: Sie haben einen Algorithmus für Thunfisch-Pizza
- Dann haben Sie auch einen Algorithmus für
 - Pizza Hawaii
 - Pizza Thunfisch-Diablo
 - Pizza Margherita
- Diese Probleme können auf das Thunfisch-Pizza-Problem reduziert werden

Pizza Margherita Algorithmus

1. Pizza Thunfisch herstellen
2. Thunfisch entfernen

Pizza Thunfisch-Diablo Algorithmus

1. Pizza Thunfisch herstellen
2. Schärfen

Pizza Hawaii Algorithmus

1. Pizza Thunfisch herstellen
2. Thunfisch entfernen
3. Schinken hinzufügen
4. Ananas hinzufügen

JETZT UMGEKEHRT: DAS PIZZA-THUNFISCH-PROBLEM

- Die Unterhitze im Backofen ist kaputt
- Sie haben durch lange Überlegung zeigen können:
Pizza Thunfisch können Sie nicht backen

- Die Unterhitze im Backofen ist kaputt
- Sie haben durch lange Überlegung zeigen können:
Pizza Thunfisch können Sie nicht backen
- Können Sie Pizza Margherita backen?
- Können Sie Pizza Hawaii backen?

- Wenn Sie Pizza Margherita backen könnten, könnten Sie auch Pizza Thunfisch backen

Pizza Thunfisch Algorithmus (aus Pizza Margherita)

1. Pizza Margherita herstellen
2. Thunfisch hinzufügen

- Wenn Sie Pizza Hawaii backen könnten, könnten Sie auch Pizza Thunfisch backen

Pizza Thunfisch Algorithmus (aus Pizza Hawaii)

1. Pizza Hawaii herstellen
2. Ananas entfernen
3. Schinken entfernen
4. Thunfisch hinzufügen

Definition (Leerbandhalteproblem)

Das Leerbandhalteproblem ist das Problem ob eine Turing-Maschine angesetzt auf das leere Band hält.

Formal: Das Leerbandhalteproblem ist die Sprache

$$H_0 = \{w \in \{0, 1\}^* \mid M_w \text{ angesetzt auf das leere Band hält}\}$$

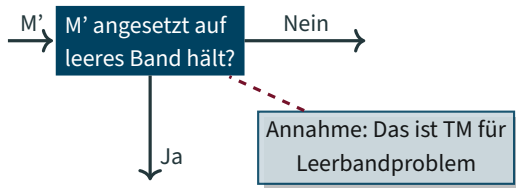
Beispiel (Anwendung)

Bleibt das Programm hängen auch wenn der Nutzer nichts eingeben muss?

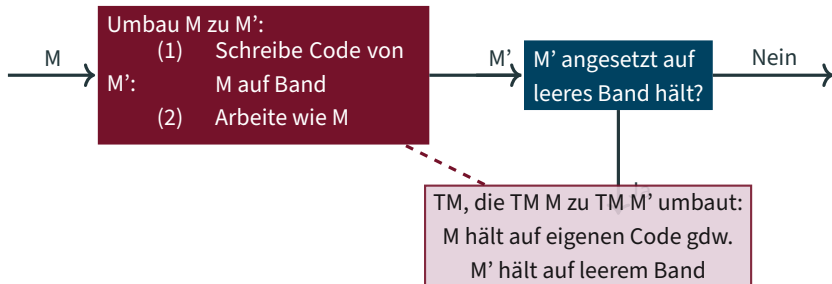
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar



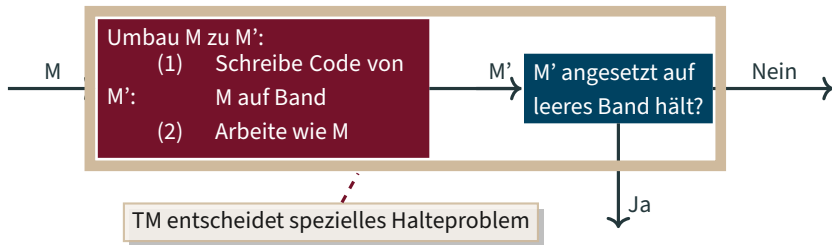
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar



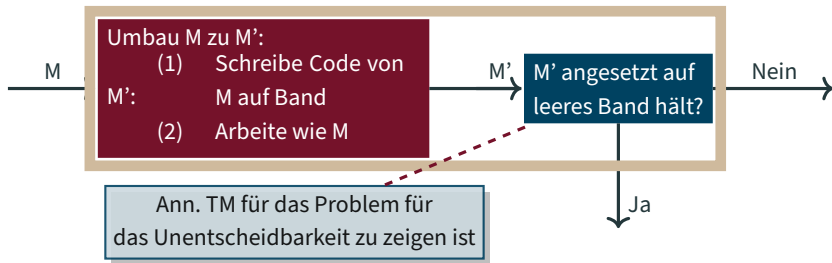
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.



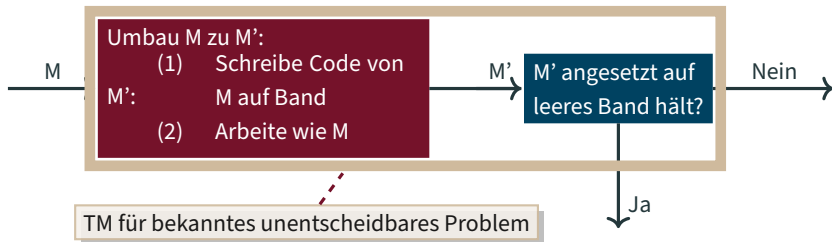
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



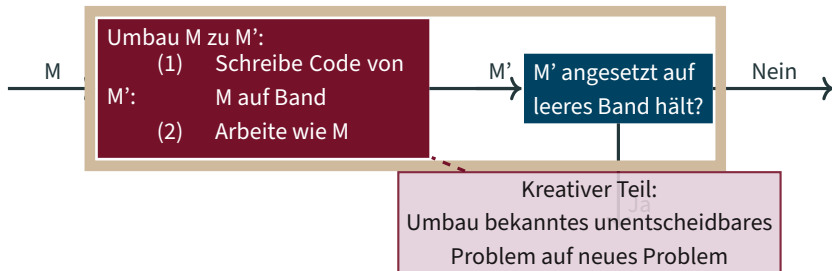
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



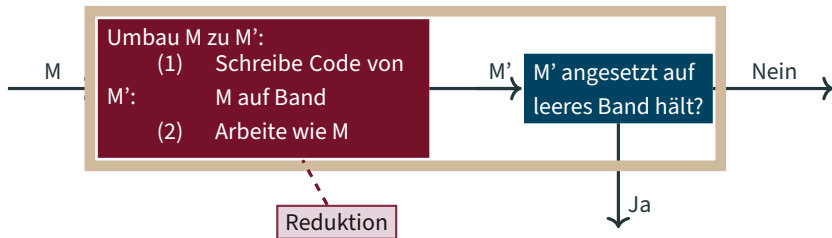
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



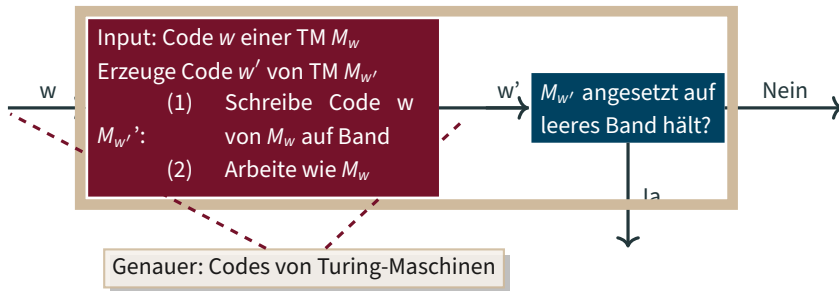
- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



- Wollen zeigen: Leerbandhalteproblem H_0 unentscheidbar
- Annahme: Leerbandhalteproblem H_0 entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar.
- Widerspruch. Also war Annahme falsch und H_0 ist unentscheidbar.



Definition (Reduktion (engl. Reducability)– formal)

Seien $A \subseteq \Sigma_1^*$ und $B \subseteq \Sigma_2^*$ Sprachen. Dann heißt A auf B reduzierbar $A \leq B$ falls es eine totale und berechenbare Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, so dass für alle $x \in \Sigma_1^*$ gilt:

$$x \in A \iff f(x) \in B$$

Definition (Reduktion (engl. Reducability)– formal)

Seien $A \subseteq \Sigma_1^*$ und $B \subseteq \Sigma_2^*$ Sprachen. Dann heißt A auf B reduzierbar $A \leq B$ falls es eine totale und berechenbare Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, so dass für alle $x \in \Sigma_1^*$ gilt:

$$x \in A \iff f(x) \in B$$

Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.

Definition (Reduktion (engl. Reducability)– formal)

Seien $A \subseteq \Sigma_1^*$ und $B \subseteq \Sigma_2^*$ Sprachen. Dann heißt A auf B reduzierbar $A \leq B$ falls es eine totale und berechenbare Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, so dass für alle $x \in \Sigma_1^*$ gilt:

$$x \in A \iff f(x) \in B$$

Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.

Beweis.

Wir wollen $x \in A$ entscheiden. Wegen $A \leq B$ gibt es eine berechenbare Funktion f mit $x \in A \iff f(x) \in B$. Wir berechnen $f(x)$. Wir entscheiden $f(x) \in B$. Das ist möglich, weil B entscheidbar ist. Nun wissen wir wegen $x \in A \iff f(x) \in B$ auch die Antwort auf die Frage ob $x \in A$ gilt. □

Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.

- Spezielle Halteproblem (A) wird auf Leerbandproblem (B) reduziert

Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.

- Spezielle Halteproblem (A) wird auf Leerbandproblem (B) reduziert
- Reduktion zeigt: Wenn das Leerbandhalteproblem (B) entscheidbar ist, dann ist auch das spezielle Halteproblem (A) entscheidbar.

Wenn B entscheidbar ist und $A \leq B$, dann ist auch A entscheidbar.

- Spezielle Halteproblem (A) wird auf Leerbandproblem (B) reduziert
- Reduktion zeigt: Wenn das Leerbandhalteproblem (B) entscheidbar ist, dann ist auch das spezielle Halteproblem (A) entscheidbar.
- Da wir wissen, dass das spezielle Halteproblem (A) unentscheidbar ist, muss dann auch das Leerbandhalteproblem (B) unentscheidbar sein.

```
/** Entscheidet ob codeM terminiert, wenn Variable input  
den Wert "" hat.*/
```

```
boolean leerbandHalteproblem(String codeM); // neues Problem
```

```
/** Entscheidet ob codeM terminiert, wenn Variable input  
den Wert codeM hat.*/
```

```
boolean speziellesHalteproblem (String codeM){
```

```
    // Reduktion
```

```
    String codeM1 = "input = "+codeM+";\n"+  
                    codeM;
```

```
    return leerbandHalteproblem(codeM1); // altes Problem
```

```
}
```

REDUKTION SPEZIELLES HALTEPROBLEM AUF LEERBANDHALTEPROBLEM

Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_{w'}$ mit Kodierung w' wie folgt:

1. Schreibe eigene Kodierung w auf das Band
2. Simuliere M_w

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält auf Eingabe w gdw. $M_{w'}$ hält auf leerem Band. (*)



REDUKTION SPEZIELLES HALTEPROBLEM AUF LEEBANDHALTEPROBLEM

Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_{w'}$ mit Kodierung w' wie folgt:

1. Schreibe eigene Kodierung w auf das Band
2. Simuliere M_w

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält auf Eingabe w gdw. $M_{w'}$ hält auf leerem Band. (*)

Angenommen das Leerbandhalteproblem ist entscheidbar. Dann ist das spezielle Halteproblem wie folgt entscheidbar: Sei M_w eine TM, konstruiere $M_{w'}$ wie in der Reduktion angegeben. Entscheide Leerbandhalteproblem für $M_{w'}$. Nutze (*).

Definition (Allgemeine Halteproblem)

Das allgemeine Halteproblem ist das Problem ob eine Turing-Maschine M angesetzt auf ein Wort x hält.

Formal: Das Halteproblem ist die Sprache

$$H = \{w\#x \mid M_w \text{ angesetzt auf } x \text{ hält}\}$$

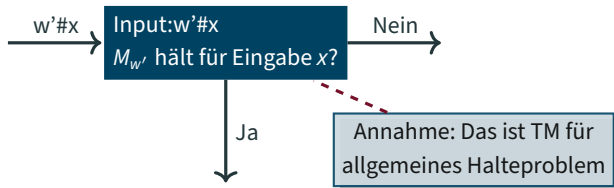
Beispiel (Anwendung)

Bleibt das Programm für eine bestimmte Eingabe hängen?

- Wollen zeigen: Allgemeines Halteproblem H unentscheidbar

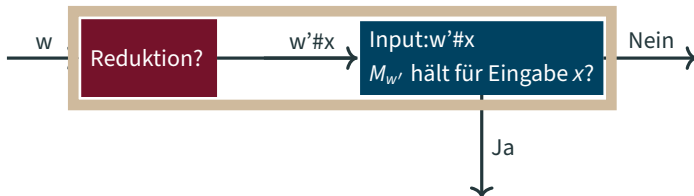
UNENTSCHEIDBARKEIT DES ALLGEMEINEN HALTEPROBLEMS

- Wollen zeigen: Allgemeines Halteproblem H unentscheidbar
- Nehmen an: Allgemeines Halteproblem H entscheidbar



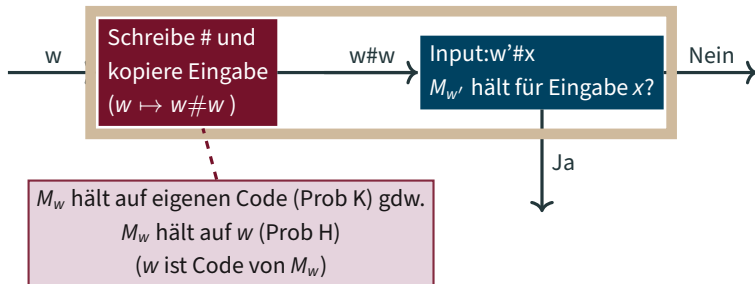
UNENTSCHEIDBARKEIT DES ALLGEMEINEN HALTEPROBLEMS

- Wollen zeigen: Allgemeines Halteproblem H unentscheidbar
- Nehmen an: Allgemeines Halteproblem H entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar



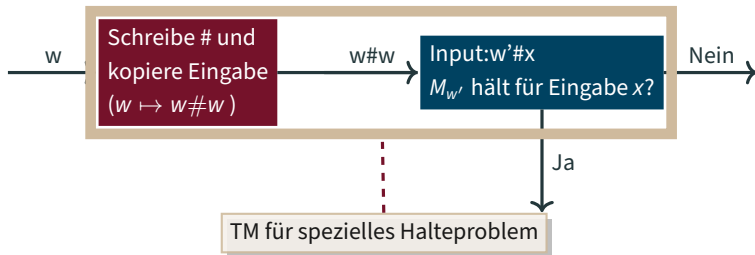
UNENTSCHEIDBARKEIT DES ALLGEMEINEN HALTEPROBLEMS

- Wollen zeigen: Allgemeines Halteproblem H unentscheidbar
- Nehmen an: Allgemeines Halteproblem H entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar



UNENTSCHEIDBARKEIT DES ALLGEMEINEN HALTEPROBLEMS

- Wollen zeigen: Allgemeines Halteproblem H unentscheidbar
- Nehmen an: Allgemeines Halteproblem H entscheidbar
- Zeigen: Unter der Annahme ist auch spezielles Halteproblem K entscheidbar
- Widerspruch. Also war die Annahme falsch und H ist unentscheidbar.



```
/** Entscheidet ob codeM terminiert, wenn Variable input  
    den Wert x hat.*/
```

```
boolean allgemeinesHalteproblem(String codeM, String x); // neu\
```

```
/** Entscheidet ob codeM terminiert, wenn Variable input  
    den Wert codeM hat.*/
```

```
boolean speziellesHalteproblem (String codeM){ //bekannt  
    return allgemeinesHalteproblem(codeM,codeM);  
}
```

Definition (Leerheitsproblem (Emptiness Problem))

Das Leerheitsproblem ist das Problem ob eine Turing-Maschine für keine Eingabe hält.

Formal: Das Leerheitsproblem ist die Sprache

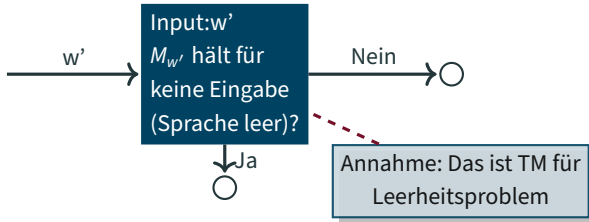
$$E = \{w \mid M_w \text{ hält für keine Eingabe}\}$$

Beispiel (Anwendung)

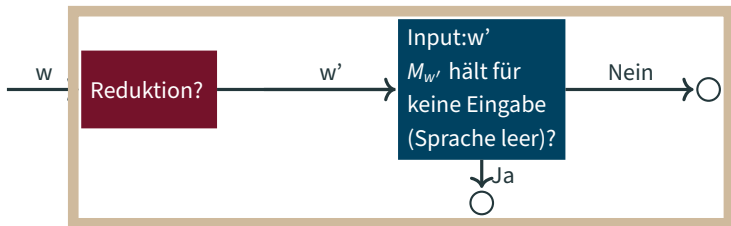
Hat der Nutzer eine Chance, eine Eingabe zu finden, so dass das Programm nicht hängen bleibt?

- Wollen zeigen: Leerheitsproblem E unentscheidbar

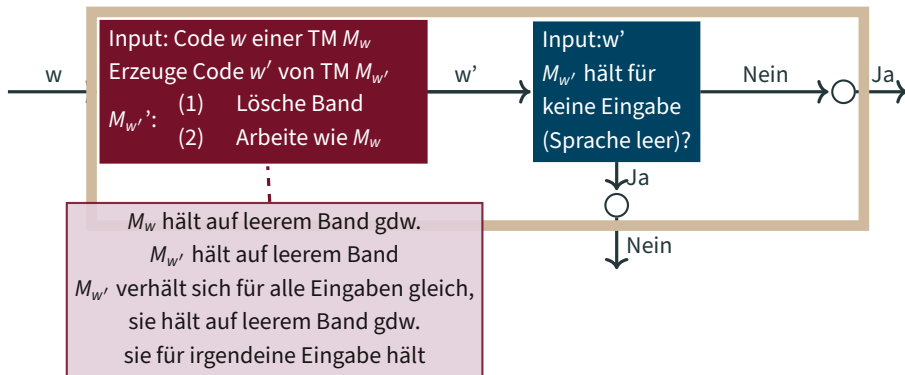
- Wollen zeigen: Leerheitsproblem E unentscheidbar
- Nehmen an: E entscheidbar



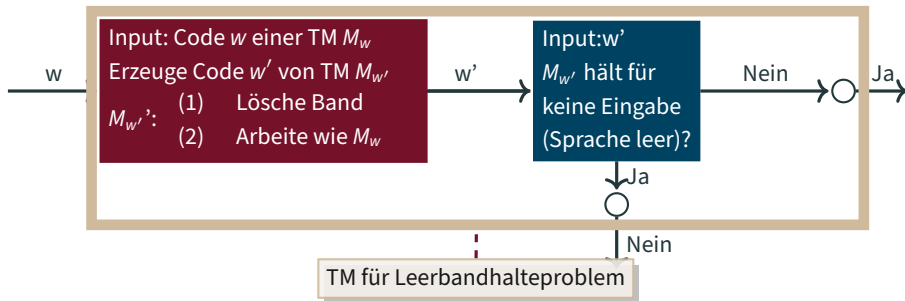
- Wollen zeigen: Leerheitsproblem E unentscheidbar
- Nehmen an: E entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerbandproblem H_0 entscheidbar



- Wollen zeigen: Leerheitsproblem E unentscheidbar
- Nehmen an: E entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerbandproblem H_0 entscheidbar



- Wollen zeigen: Leerheitsproblem E unentscheidbar
- Nehmen an: E entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerbandproblem H_0 entscheidbar



```
/** Liefert true gdw codeM für keine Belegung der  
    Variablen input terminiert. */
```

```
boolean leerheitsProblem(String codeM); // neu
```

```
/** Entscheidet ob codeM terminiert, wenn Variable input  
    den Wert "" hat.*/
```

```
boolean leerbandHalteproblem (String codeM){//bekannt
```

```
// Reduktion
```

```
String codeM1 = "input = \"\";\n"+  
                codeM;
```

```
return !leerheitsproblem(codeM1);
```

```
}
```

Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_{w'}$ mit Kodierung w' wie folgt:

1. Lösche das Band
2. Simuliere M_w

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält auf leerem Band gdw. $M_{w'}$ hält für eine Eingabe
gdw. es gilt nicht: $M_{w'}$ hält für keine Eingabe (*)



Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_{w'}$ mit Kodierung w' wie folgt:

1. Lösche das Band
2. Simuliere M_w

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält auf leerem Band gdw. $M_{w'}$ hält für eine Eingabe
gdw. es gilt nicht: $M_{w'}$ hält für keine Eingabe (*)

Angenommen das Leerheitsproblem ist entscheidbar. Dann ist das Leerbandhalteproblem wie folgt entscheidbar: Sei M_w eine TM, konstruiere $M_{w'}$ wie in der Reduktion angegeben.

Entscheide Leerheitsproblem für $M_{w'}$. Nutze (*).



Definition (Äquivalenzproblem)

Das Äquivalenzproblem ist das Problem ob zwei Turing-Maschine die gleiche Sprache akzeptieren.

Formal: Das Äquivalenzproblem ist die Sprache

$$EQ = \{w_1 \# w_2 \mid M_{w_1} \text{ hält für } w \text{ gdw. } M_{w_2} \text{ hält für } w\}$$

Beispiel (Anwendung)

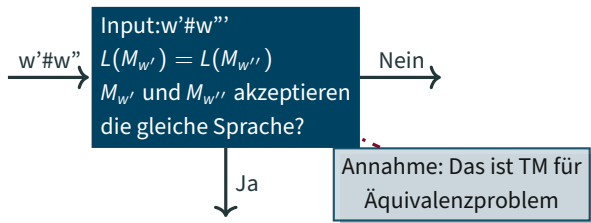
- Refactoring erfolgreich?
- Programm erfüllt Spezifikation?

UNENTSCHEIDBARKEIT DES ÄQUIVALENZPROBLEMS

- Wollen zeigen: Äquivalenzproblem EQ unentscheidbar

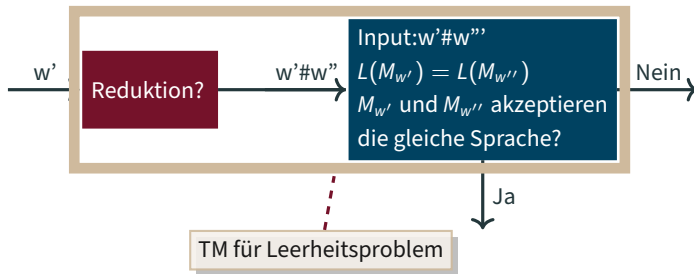
UNENTSCHEIDBARKEIT DES ÄQUIVALENZPROBLEMS

- Wollen zeigen: Äquivalenzproblem EQ unentscheidbar
- Nehmen an: EQ entscheidbar



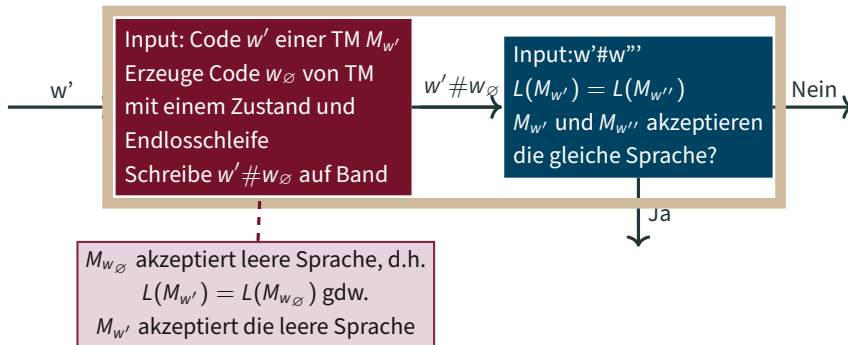
UNENTSCHEIDBARKEIT DES ÄQUIVALENZPROBLEMS

- Wollen zeigen: Äquivalenzproblem EQ unentscheidbar
- Nehmen an: EQ entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerheitsproblem E entscheidbar



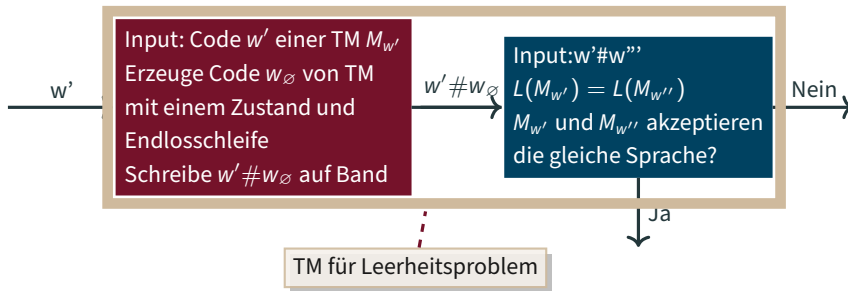
UNENTSCHEIDBARKEIT DES ÄQUIVALENZPROBLEMS

- Wollen zeigen: Äquivalenzproblem EQ unentscheidbar
- Nehmen an: EQ entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerheitsproblem E entscheidbar



UNENTSCHEIDBARKEIT DES ÄQUIVALENZPROBLEMS

- Wollen zeigen: Äquivalenzproblem EQ unentscheidbar
- Nehmen an: EQ entscheidbar
- Zeigen: Unter der Annahme ist auch das Leerheitsproblem E entscheidbar



```
/** Liefert true gdw für alle Werte von input  
    codeM1 terminiert genau dann wenn codeM2 terminiert*/  
boolean aquivalenzProblem(String codeM1, String codeM2); // neu  
  
/** Liefert true gdw codeM für keine Belegung der  
    Variablen input terminiert. */  
boolean leerheitsProblem(String codeM){ // bekannt  
    // Reduktion  
    String codeM1 = "while (true){};"  
    return aequivalenzproblem(codeM,codeM1);  
}
```

Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_\emptyset$ mit Kodierung $w_\emptyset$ wie folgt:

1. TM mit einem Zustand und Endlosschleife

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält für keine Eingabe gdw. M_w und $M_\emptyset$ akzeptieren die gleiche Sprache. (*)



Beweis.

Sei w die Kodierung einer TM M_w . Konstruiere Turing-Maschine $M_\emptyset$ mit Kodierung $w_\emptyset$ wie folgt:

1. TM mit einem Zustand und Endlosschleife

Diese Konstruktion ist effektiv berechenbar und es gilt dann

M_w hält für keine Eingabe gdw. M_w und $M_\emptyset$ akzeptieren die gleiche Sprache. (*)

Ann. das Äquivalenzproblem ist entscheidbar. Entscheide Leerheitsproblem wie folgt: Sei M_w eine TM, konstruiere $M_\emptyset$ wie in der Reduktion angegeben. Entscheide Äquivalenzproblem für M_w und $M_\emptyset$. Nutze (*).



Für Programme ist also unentscheidbar

- ob sie halten
- ob sie für irgendeine Eingabe in eine Endlosschleife laufen
- ob zwei Programme gleich sind
- ...